

Desenvolvimento de um Serviço de Metadata Compatível com Cloud-Init para Incus



Gustavo Santos

Orientador: Prof. Maycon L. M. Peixoto





Agenda

1. Introdução
2. Trabalhos Relacionados
3. Proposta
4. Resultados
5. Conclusão

Introdução

Contexto

- Nuvens públicas (AWS, GCP, Azure) provisionam instâncias de forma programática e automatizada.
- Peça central: *Instance Metadata Service* (IMDS), acessível pelo endereço *link-local* 169.254.169.254 [1].
- Cada instância descobre a própria configuração no *boot*: hostname, endereços IP, chaves SSH, *user-data*.
- O **cloud-init** [3] consome esses dados e é o padrão da indústria para inicialização automatizada de instâncias.

Problema

- O **Incus** [11] (*fork* comunitário do LXD) não oferece um serviço de metadados HTTP nativo.
- A entrega de configuração existente (*datasource* LXD via *socket* `/dev/incus/sock`) sofre uma **condição de corrida**: o *Incus agent* inicia tarde demais para os primeiros estágios do cloud-init em VMs.
- Consequências:
 - configuração manual ou *scripts* ad hoc;
 - sem portabilidade de configurações entre nuvem pública e privada;
 - OpenStack resolve, mas exige a plataforma completa — desproporcional para *homelabs*, borda e nuvens privadas pequenas.

Objetivo e Contribuições

Objetivo: projetar e implementar um serviço de metadados compatível com cloud-init para o Incus, leve e autônomo.

1. Análise das implementações de serviços de metadados em nuvens públicas e privadas e dos requisitos mínimos de compatibilidade com o cloud-init.
2. Projeto e implementação do serviço: API REST, sincronização de instâncias orientada a eventos, persistência e alta disponibilidade.
3. Avaliação em ambiente Incus real: corretude funcional, desempenho, escalabilidade e *failover*.

Trabalhos Relacionados

Serviços de Metadados Existentes

- **AWS IMDS** [1]: API HTTP em 169.254.169.254; IMDSv2 adiciona autenticação por sessão contra SSRF.
- **GCP Metadata Server** [8]: exige o cabeçalho Metadata-Flavor: Google; escopos de projeto e de instância.
- **OpenStack Metadata** [13]: compatível com a API EC2, mas fortemente acoplado ao Neutron e ao Nova — não executa de forma autônoma.
- **Datasource NoCloud** [4]: obtém dados de uma URL base (seedfrom) consultando /meta-data, /user-data, /vendor-data e /network-config — exige apenas instance-id e local-hostname.
- **RAFT** [12]: consenso distribuído compreensível (eleição de líder + replicação de log); base da alta disponibilidade.

Comparação entre as Abordagens

Solução	Autônomo	Compat. cloud-init	Contêineres	VMs	Alta Disp.	Cód. Aberto
AWS IMDS	X	✓	X	✓	✓	X
GCP Metadata	X	✓	X	✓	✓	X
OpenStack Metadata	X	✓	X	✓	✓	✓
LXD/Incus (atual)	✓	X	✓	✓	X	✓
Este trabalho	✓	✓	✓	✓	✓	✓

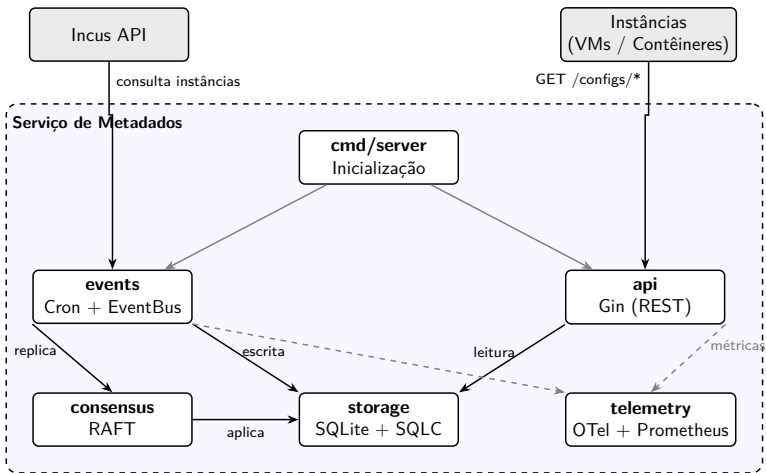
Lacuna: não há serviço de metadados leve e autônomo que implemente o protocolo HTTP esperado pelo cloud-init para o Incus.

Proposta

Visão Geral

- Serviço de metadados compatível com cloud-init para instâncias Incus (contêineres e VMs).
- API REST consultada pelas instâncias durante o *boot*, seguindo o modelo do *datasource* NoCloud HTTP.
- A instância é identificada pelo **endereço IP de origem** da requisição — sem agentes ou configuração dentro da instância.
- Sincronização orientada a eventos mantém o estado consistente com o hipervisor Incus.
- Implantação autônoma: um único binário Go, banco embutido, sem dependências externas.

Arquitetura



Sincronização de Instâncias

Ciclo periódico e orientado a eventos (*pub/sub*):

1. *Cron* a cada 10 s publica o evento `instances.sync`.
2. O *handler* consulta a API do Incus e lista as instâncias ativas.
3. Para cada instância, um evento `instance.sync` é publicado — processamento desacoplado e paralelo.
4. Para cada instância, o serviço:
 - extrai rede (IPv4, IPv6, MAC) do estado reportado pelo Incus;
 - persiste metadados cloud-init (`instance-id`, `hostname`, chaves SSH, *placement*);
 - armazena `user-data` e `vendor-data` das chaves `cloud-init.*` do Incus;
 - gera `network-config` (Networking Config v2 / Netplan) quando não definida pelo administrador.

Endpoints da API

Endpoints públicos, no modelo do *datasource* NoCloud HTTP [4]:

- GET `/configs/meta-data` — identidade da instância (`instance-id`, `hostname`, `IPs`, `public-keys`).
- GET `/configs/meta-data/:key` — campo específico dos metadados.
- GET `/configs/user-data` — *scripts/cloud-config* do usuário.
- GET `/configs/vendor-data` — por instância, com *fallback* global.
- GET `/configs/network-config` — definida pelo administrador ou gerada automaticamente.

Endpoints internos (`/internal/*`): CRUD administrativo de `vendor-data` — não expostos às instâncias.

Descoberta do Serviço pelas Instâncias

- Mesma convenção das nuvens públicas: endereço *link-local* 169.254.169.254 adicionado à *bridge* do Incus (`incusbr0`).
- Regra NAT redireciona 169.254.169.254:80 para a porta do serviço — nenhuma configuração dentro das instâncias.
- *Datasource* NoCloud em modo rede (`dsmode: net`), com `seedfrom` apontando para `/configs/` no perfil do Incus.

Alta Disponibilidade via RAFT

Nó único = ponto único de falha. Solução: consenso distribuído RAFT [12] com `hashicorp/raft` [9].

- **Eleição de líder:** apenas o líder sincroniza com o Incus e propõe escritas; falha do líder dispara nova eleição automática.
- **Replicação de log + FSM:** cada escrita vira um comando tipado, confirmado por maioria e aplicado ao SQLite local de cada nó.
- **Persistência:** log RAFT em BoltDB — sem banco externo.
- **Leituras em qualquer nó:** todos mantêm cópia local do estado, distribuindo a carga de consultas.
- Ativação opcional por variáveis de ambiente (`RAFT_ENABLED`, `NODE_ID`, `PEERS`, ...).

Tecnologias Utilizadas

- **Go** — binário estático único; goroutines para o processamento paralelo de eventos.
- **Gin** [7] — framework HTTP de alto desempenho; separa rotas públicas e internas.
- **SQLite + SQLC** [10] — banco embutido; queries SQL com tipagem estática gerada.
- **GoEventBus** — barramento *pub/sub* que desacopla agendamento e sincronização.
- **gocron** — agendamento do ciclo de sincronização (10 s).
- **hashicorp/raft** [9] — consenso distribuído.
- **OpenTelemetry + Prometheus** [5, 6] — métricas de sincronização e de API em `/metrics`.

Resultados

Metodologia de Avaliação

- Ambiente: GCE e2-standard-4 (4 vCPUs, 15 GiB), Ubuntu 24.04 LTS, Incus 6.0.0; serviço no mesmo host das instâncias.
- Cargas de 5 a 200 contêineres simultâneos com cloud-init apontando para o serviço.
- Latência medida com hey: 5000 requisições, concorrência 50; percentis p50/p95/p99.
- **Achado prático:** em nuvem pública, 169.254.169.254 já é usado pelo provedor — o *datasource* foi apontado para o *gateway* da *bridge* (10.10.10.1:8080).

Avaliação Funcional

Propriedade	Resultado
Isolamento de metadados	Aprovado
Isolamento sob falsificação de IP (X-Forwarded-For)	Aprovado
Execução de <i>user-data</i>	Aprovado
Entrega de <i>network-config</i>	Aprovado
Propagação de estado após reinício	Aprovado

Inicialização real: cloud-init selecionou o *datasource* NoCloud de rede, obteve os dados (HTTP 200) e concluiu com `cloud-init status: done` — compatibilidade de ponta a ponta.

Desempenho: Latência da API

Endpoint	p50 (ms)	p95 (ms)	p99 (ms)
/meta-data	13,2	53,7	83,2
/user-data	18,9	53,6	72,4
/network-config	11,4	47,2	72,4
/vendor-data	17,0	48,4	68,9

- Medianas de 11–19 ms; p99 abaixo de 90 ms — compatível com as janelas de tempo do *boot*.
- Latência de sincronização (criação no Incus → metadados disponíveis): $\approx 20,5$ s (≈ 2 ciclos de *polling*).

Desempenho: Escalabilidade

Instâncias	p50 (ms)	p95 (ms)	p99 (ms)	Erros	Mem. (MB)
5	16,2	60,3	92,4	0,0%	1152
25	12,4	51,8	77,2	0,0%	1373
50	12,9	52,0	80,8	0,0%	1733
100	14,4	58,6	92,1	0,0%	1763
200	12,8	54,2	87,2	0,0%	1711

- Hipótese inicial: degradação por serialização de escrita do SQLite.
- **Não corroborada** até 200 instâncias: latência estável e zero erros — carga dominada por leituras; escritas a cada 10 s absorvidas pelo modo WAL.

Alta Disponibilidade (*cluster* de 3 nós)

Propriedade	Resultado
Formação do <i>cluster</i>	1 líder + 2 seguidores
Replicação de estado	instância presente nos 3 nós
Reeleição após falha do líder	2,40 s
Disponibilidade após <i>failover</i>	preservada
Reintegração de nó	reingressa como seguidor

- Topologia avaliada: réplicas sobre um Incus compartilhado — reconciliação correta e *failover* sem perda de dados.
- Um Incus por nó (borda distribuída) exigiria reconciliação por nó de origem.

Discussão e Limitações

- Cobre os campos essenciais dos módulos mais comuns do cloud-init (`set_hostname`, `users-groups`, `write-files`, `runcmd`, rede).
- **Limitações:**
 - SQLite serializa escritas — potencial gargalo bem acima da escala avaliada;
 - subconjunto dos campos de AWS/GCP (sem tipo de instância, faturamento);
 - garantias de *failover* pressupõem a topologia de Incus compartilhado.
- **Ameaças à validade:** host único nos testes de desempenho; contêineres de sistema (não VMs completas); janela de inconsistência do *polling*.

Conclusão

Conclusão

- Serviço de metadados compatível com cloud-init para o Incus: preenche a lacuna de um endpoint HTTP autônomo em infraestrutura privada.
- Identificação por IP de origem: operação transparente, sem agentes nas instâncias.
- Validado em ambiente real: *boots* completos com sucesso, latências de dezenas de milissegundos, escalabilidade estável até 200 instâncias e reeleição de líder em $\approx 2,4$ s.
- Código aberto, binário único, implantação simples — adequado a *homelabs*, borda e nuvens privadas de pequena escala.

Trabalhos Futuros

1. Suporte à convenção 169.254.169.254 via *namespace* de rede ou *proxy* reverso.
2. Ampliação dos módulos e campos do cloud-init suportados.
3. Autenticação baseada em sessões inspirada no IMDSv2 [2] contra SSRF.
4. Substituição do *polling* pela API de eventos do Incus (sincronização em tempo real).
5. *Benchmarks* em cenários de maior escala.



Obrigado!

Perguntas?

`gustavoafs@ufba.br`

Referências

- [1] Amazon Web Services. Instance metadata and user data — Amazon EC2 documentation, 2024. Accessed: 2025-03-13.
- [2] Amazon Web Services. Use IMDSv2 — Amazon EC2 documentation, 2024. Accessed: 2025-03-13.
- [3] Canonical Ltd. cloud-init documentation, 2024. Accessed: 2025-03-13.
- [4] Canonical Ltd. Nocloud — cloud-init documentation, 2024. Accessed: 2025-03-13.
- [5] Cloud Native Computing Foundation. OpenTelemetry — an observability framework for cloud-native software, 2024. Accessed: 2025-03-13.
- [6] Cloud Native Computing Foundation. Prometheus — monitoring system & time series database, 2024. Accessed: 2025-03-13.
- [7] Gin-Gonic Contributors. Gin web framework, 2024. Accessed: 2025-03-13.

Referências (cont.)

- [8] Google Cloud. About VM metadata — Google Cloud documentation, 2024. Accessed: 2025-03-13.
- [9] HashiCorp. hashicorp/raft — golang implementation of the raft consensus protocol, 2024. Accessed: 2025-03-13.
- [10] D. Richard Hipp. SQLite — small. fast. reliable. choose any three., 2024. Accessed: 2025-03-13.
- [11] Linux Containers. Incus — system container and virtual machine manager, 2024. Accessed: 2025-03-13.
- [12] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In *2014 USENIX Annual Technical Conference (USENIX ATC 14)*, pages 305–319, 2014.
- [13] OpenStack Foundation. Metadata service — OpenStack documentation, 2024. Accessed: 2025-03-13.